Caesar Cipher

```
import java.util.*;
public class Main {
   static String caesar(String s, int k) {
      String r = "";
      for (char c : s.toCharArray()) {
         if (Character.isLetter(c))
            r += (char)('A' + (c - 'A' + k + 26) % 26);
         else r += c;
      }
      return r;
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter plaintext: ");
      String s = sc.next().toUpperCase();
      System.out.print("Enter key: ");
      int k = sc.nextInt();
      String e = caesar(s, k);
      System.out.println("Encrypted: " + e);
      System.out.println("Decrypted: " + caesar(e, -k));
   }
}
```

Play fair

```
package cns_f;
import java.util.*;

public class PlayfairCipher {

   static char[][] matrix = {
      {'M','O','N','A','R'},
      {'C','H','Y','B','D'},
      {'E','F','G','I','K'},
      {'L','P','Q','S','T'},
      {'U','V','W','X','Z'}
   };

   static int[] findPosition(char ch) {
      if (ch == 'J') ch = 'I';

      for (int i = 0; i < 5; i++)
         for (int j = 0; j < 5; j++)
            if (matrix[i][j] == ch)
               return new int[]{i, j};

      return null;
   }
```

```
static String encrypt(String text) {
   String result = "";

   for (int i = 0; i < text.length(); i += 2) {
      int[] p1 = findPosition(text.charAt(i));
      int[] p2 = findPosition(text.charAt(i + 1));

      int r1 = p1[0], c1 = p1[1];
      int r2 = p2[0], c2 = p2[1];

      if (r1 == r2) {
         result += matrix[r1][(c1 + 1) % 5];
         result += matrix[r2][(c2 + 1) % 5];
      } else if (c1 == c2) {
         result += matrix[(r1 + 1) % 5][c1];
         result += matrix[(r2 + 1) % 5][c2];
      } else {
         result += matrix[r1][c2];
         result += matrix[r2][c1];
      }
   }
   return result;
}

static String decrypt(String text) {
   String result = "";

   for (int i = 0; i < text.length(); i += 2) {
      int[] p1 = findPosition(text.charAt(i));
      int[] p2 = findPosition(text.charAt(i + 1));

      int r1 = p1[0], c1 = p1[1];
      int r2 = p2[0], c2 = p2[1];

      if (r1 == r2) {
         result += matrix[r1][(c1 + 4) % 5];
         result += matrix[r2][(c2 + 4) % 5];
      } else if (c1 == c2) {
         result += matrix[(r1 + 4) % 5][c1];
         result += matrix[(r2 + 4) % 5][c2];
      } else {
         result += matrix[r1][c2];
         result += matrix[r2][c1];
      }
   }
   return result;
}
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter plaintext (even number of letters): ");
        String text = sc.next().toUpperCase();

        if (text.length() % 2 != 0) {
            System.out.println("Text length must be even!");
            sc.close();
            return;
        }

        String encrypted = encrypt(text);
        System.out.println("Encrypted Text: " + encrypted);
        System.out.println("Decrypted Text: " + decrypt(encrypted));

        sc.close();
    }
}
```

Hill Cipher

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] k = {{3, 3}, {2, 5}};
        int[][] inv = {{15, 17}, {20, 9}};

        System.out.print("Enter 2-letter plaintext: ");
        String s = sc.next().toUpperCase();
        int a = s.charAt(0)-'A', b = s.charAt(1)-'A';

        int x = (3*a + 3*b) % 26;
        int y = (2*a + 5*b) % 26;
        String e = "" + (char)(x+'A') + (char)(y+'A');
        System.out.println("Encrypted: " + e);

        int p = (15*x + 17*y) % 26;
        int q = (20*x + 9*y) % 26;
        System.out.println("Decrypted: " +
            (char)(p+'A') + "" + (char)(q+'A'));
    }
}
```

Rail Fence Cipher

```
import java.util.*;

class RailFence {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter message: ");
      String s = sc.nextLine().replace(" ", "");
      String enc = "", dec = "";

      for (int i = 0; i < s.length(); i += 2)
         enc += s.charAt(i);
      for (int i = 1; i < s.length(); i += 2)
         enc += s.charAt(i);

      int mid = (s.length() + 1) / 2;
      dec = "";
      for (int i = 0; i < mid; i++) {
         dec += enc.charAt(i);
         if (i + mid < enc.length())
            dec += enc.charAt(i + mid);
      }

      System.out.println("Encrypted: " + enc);
      System.out.println("Decrypted: " + dec);
   }
}
```

Row & Column Transformation (Columnar Transposition)

```
import java.util.*;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter message: ");
      String s = sc.nextLine();

      int n = 4;
      while (s.length() % n != 0) s += "X";

      String enc = "", dec = "";
      int rows = s.length() / n;
      char[][] a = new char[rows][n];

      for (int i = 0; i < s.length(); i++)
         a[i / n][i % n] = s.charAt(i);
```

```
      for (int c = 0; c < n; c++)
         for (int r = 0; r < rows; r++)
            enc += a[r][c];

      int i=0;
      for (int c = 0; c < n; c++)
         for (int r = 0; r < rows; r++)
            a[r][c] = enc.charAt(i++);

      for (int r = 0; r < rows; r++)
         for (int c = 0; c < n; c++)
            dec += a[r][c];

      System.out.println("Encrypted: " + enc);
      System.out.println("Decrypted: " + dec.replaceAll("X",""));
   }
}
```

Image Steganography

```
import java.awt.image.BufferedImage;
import java.io.*;
import javax.imageio.ImageIO;
import java.util.*;

public class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);

      BufferedImage img = ImageIO.read(new File("input.png"));

      System.out.print("Enter text: ");
      String msg = sc.nextLine();
      byte[] b = msg.getBytes();

      int k = 0;
      for (int i = 0; i < img.getWidth() && k < b.length; i++)
         for (int j = 0; j < img.getHeight() && k < b.length; j++) {
            int p = img.getRGB(i, j);
            p = (p & 0xFFFFFF00) | (b[k] & 0xFF);
            img.setRGB(i, j, p);
            k++;
         }

      ImageIO.write(img, "png", new File("output.png"));

      BufferedImage out = ImageIO.read(new File("output.png"));
```

```
      String s = "";
      k = 0;

      for (int i = 0; i < out.getWidth() && k < b.length; i++)
         for (int j = 0; j < out.getHeight() && k < b.length; j++) {
            int p = out.getRGB(i, j);
            s += (char) (p & 0xFF);
            k++;
         }

      System.out.println("Hidden Message: " + msg);
      System.out.println("Retrieved Message: " + s);
      sc.close();
   }
}
```

DES Algorithm

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;

class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter text: ");
      String s = sc.nextLine();

      SecretKey key = new SecretKeySpec(
         "12345678".getBytes(), "DES");

      Cipher c = Cipher.getInstance("DES");

      c.init(Cipher.ENCRYPT_MODE, key);
      byte[] e = c.doFinal(s.getBytes());
      System.out.println("Encrypted: " +
         Base64.getEncoder().encodeToString(e));

      c.init(Cipher.DECRYPT_MODE, key);
      System.out.println("Decrypted: " +
         new String(c.doFinal(e)));
   }
}
```

AES

```
import javax.crypto.*;
import javax.crypto.spec.*;
```

```
import java.util.*;

class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter text: ");
      String s = sc.nextLine();

      SecretKey key = new SecretKeySpec(
         "1234567890123456".getBytes(), "AES");

      Cipher c = Cipher.getInstance("AES");

      c.init(Cipher.ENCRYPT_MODE, key);
      byte[] e = c.doFinal(s.getBytes());
      System.out.println("Encrypted: " +
         Base64.getEncoder().encodeToString(e));

      c.init(Cipher.DECRYPT_MODE, key);
      System.out.println("Decrypted: " +
         new String(c.doFinal(e)));
   }
}
```

Electronic Codebook (ECB)

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;

class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter text: ");
      String s = sc.nextLine();

      SecretKey key = new SecretKeySpec(
         "1234567890123456".getBytes(), "AES");
      Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");

      c.init(Cipher.ENCRYPT_MODE, key);
      byte[] e = c.doFinal(s.getBytes());
      System.out.println("Encrypted: " + Base64.getEncoder().encodeToString(e));

      c.init(Cipher.DECRYPT_MODE, key);
      System.out.println("Decrypted: " + new String(c.doFinal(e)));
   }
```

```
}
```

CBC

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;

class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter text: ");
      String s = sc.nextLine();

      SecretKey key = new SecretKeySpec(
         "1234567890123456".getBytes(), "AES");
      IvParameterSpec iv = new IvParameterSpec(
         "1234567890123456".getBytes());
      Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");

      c.init(Cipher.ENCRYPT_MODE, key, iv);
      byte[] e = c.doFinal(s.getBytes());
      System.out.println("Encrypted: " + Base64.getEncoder().encodeToString(e));

      c.init(Cipher.DECRYPT_MODE, key, iv);
      System.out.println("Decrypted: " + new String(c.doFinal(e)));
   }
}
```

RC4 Algorithm

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;

class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter text: ");
      String s = sc.nextLine();

      SecretKeySpec key = new SecretKeySpec("SECRET".getBytes(), "RC4");
      Cipher c = Cipher.getInstance("RC4");

      c.init(Cipher.ENCRYPT_MODE, key);
      byte[] e = c.doFinal(s.getBytes());
      System.out.println("Encrypted: " + Base64.getEncoder().encodeToString(e));
      c.init(Cipher.DECRYPT_MODE, key);
```

```
        System.out.println("Decrypted: " + new String(c.doFinal(e)));

        sc.close();
    }
}
```

IDEA

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;
import java.security.*;
import org.bouncycastle.jce.provider.BouncyCastleProvider;

class Main {
    public static void main(String[] args) throws Exception {
        Security.addProvider(new BouncyCastleProvider());

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter text: ");
        String s = sc.nextLine();

        SecretKey key = new SecretKeySpec(
            "1234567890123456".getBytes(), "IDEA");

        Cipher c = Cipher.getInstance("IDEA/ECB/PKCS7Padding", "BC");

        c.init(Cipher.ENCRYPT_MODE, key);
        byte[] e = c.doFinal(s.getBytes());
        System.out.println("Encrypted: " +
            Base64.getEncoder().encodeToString(e));

        c.init(Cipher.DECRYPT_MODE, key);
        System.out.println("Decrypted: " + new String(c.doFinal(e)));

        sc.close();
    }
}
```

RSA

```
import javax.crypto.Cipher;
import java.security.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter plaintext: ");
        String s = sc.nextLine();
```

```
        KeyPairGenerator g = KeyPairGenerator.getInstance("RSA");
        g.initialize(2048);
        KeyPair kp = g.generateKeyPair();
        Cipher c = Cipher.getInstance("RSA");
        c.init(Cipher.ENCRYPT_MODE, kp.getPublic());
        byte[] e = c.doFinal(s.getBytes());
        System.out.println("Encrypted: " +
            Base64.getEncoder().encodeToString(e));
        c.init(Cipher.DECRYPT_MODE, kp.getPrivate());
        System.out.println("Decrypted: " + new String(c.doFinal(e)));
    }
}
```

Diffie-Hellman Key Exchange

```
class Main {
    static int power(int x, int y, int p) {
        int r = 1;
        for (int i = 0; i < y; i++)
            r = (r * x) % p;
        return r;
    }

    public static void main(String[] args) {
        int p = 31, g = 23, a = 6, b = 15;

        int A = power(g, a, p);
        int B = power(g, b, p);

        int k1 = power(B, a, p);
        int k2 = power(A, b, p);

        System.out.println("Alice Public Key: " + A);
        System.out.println("Bob Public Key: " + B);
        System.out.println("Alice Shared Key: " + k1);
        System.out.println("Bob Shared Key: " + k2);
    }
}
```

message digest of a text using the SHA-512 algorithm.

```
import java.security.*;
import java.util.*;

class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String s = sc.nextLine();
```

```
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        byte[] d = md.digest(s.getBytes());

        for (byte b : d)
            System.out.printf("%02x", b);

        sc.close();
    }
}
```

Implement HMAC to ensure message integrity and authentication.

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();
        System.out.print("Enter key: ");
        String key = sc.nextLine();

        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key.getBytes(), "HmacSHA256"));
        byte[] h = mac.doFinal(msg.getBytes());

        for (byte b : h)
            System.out.printf("%02x", b);
        System.out.println();
    }
}
```

Digital Signature

```
import java.security.*;
import java.util.*;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();

        KeyPairGenerator g = KeyPairGenerator.getInstance("RSA");
        g.initialize(2048);
        KeyPair kp = g.generateKeyPair();
```

```
        Signature s = Signature.getInstance("SHA256withRSA");
        s.initSign(kp.getPrivate());
        s.update(msg.getBytes());
        byte[] sign = s.sign();

        s.initVerify(kp.getPublic());
        s.update(msg.getBytes());
        System.out.println("Signature verified: " + s.verify(sign));
    }
}
```

Simulate PGP

```
import java.util.*;
import java.io.*;
import java.util.zip.*;
import java.security.*;
import javax.crypto.*;

class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String msg = sc.nextLine();

        // Digital signature
        KeyPair kp = KeyPairGenerator.getInstance("RSA").generateKeyPair();
        Signature s = Signature.getInstance("SHA256withRSA");
        s.initSign(kp.getPrivate());
        s.update(msg.getBytes());
        byte[] sign = s.sign();

        // Combine message and signature
        ByteArrayOutputStream b = new ByteArrayOutputStream();
        b.write(msg.getBytes());
        b.write(sign);

        // Compress
        ByteArrayOutputStream z = new ByteArrayOutputStream();
        GZIPOutputStream gz = new GZIPOutputStream(z);
        gz.write(b.toByteArray());
        gz.close();
        byte[] data = z.toByteArray();

        // Encrypt compressed data
        KeyGenerator kg = KeyGenerator.getInstance("AES");
        kg.init(128);
        SecretKey key = kg.generateKey();
```

```
        Cipher c = Cipher.getInstance("AES");
        c.init(Cipher.ENCRYPT_MODE, key);
        byte[] enc = c.doFinal(data);
        System.out.println("Encrypted PGP message: " +
            Base64.getEncoder().encodeToString(enc));

        // Decrypt
        c.init(Cipher.DECRYPT_MODE, key);
        byte[] dec = c.doFinal(enc);
        System.out.println("Decrypted and compressed data recovered: " + dec.length + " bytes");

        sc.close();
    }
}
```